



Introduction to Scripting

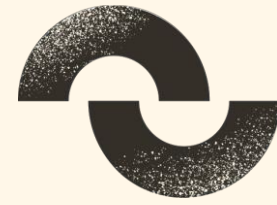
What is Scripting?

A **script** is a **set of instructions** created to perform a task.

Scripting, therefore, is the act of writing the script.

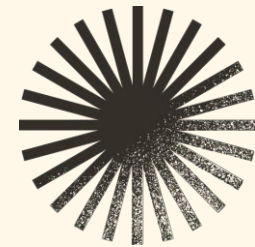
Cybersecurity professionals write scripts to **automate tasks**, **solve certain problems**, create **reproduceable findings**, etc.

Why Should You Learn Scripting?



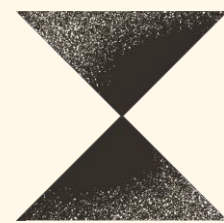
Task Automation

- Creates reliable, repeatable processes
- Reduces the likelihood of human error
- Saves time & effort



Valuable Skill

- Adds another tool to solve problems
- Potential interview questions for InfoSec & Cyber
- Makes your life easier



Strengthens Fundamentals

- Deepens your understanding of how tools work
- Helps with troubleshooting if/when tools fail
- Knowing your way around a terminal unlocks

GUI vs CLI

Graphical User Interfaces (GUI) are incredibly **convenient** and **easy-to-understand**.

→ W.Y.S.I.W.Y.G – What you see is what you get

However, you are then limited to the **GUI** and nothing more

→ W.Y.S.I.A.Y.G – What you see is *all you'll get*

GUI vs CLI

Command-line interfaces (CLIs) give you much more **control**, **flexibility**, and **speed**.

→ If you plan on working in **Cybersecurity**, you may need to **process terabytes** of information (log files, error messages, scanning outputs).

It does require some learning but is incredibly **powerful** once mastered.

UNIX Philosophy

“Do one thing and do it well”

“More shell, less egg”

- Read a file of text, determine the n most frequently used words, and print out a sorted list of those words along with their frequencies.
- Knuth made a 10-page PASCAL program that was beautifully documented, used clever data structures/algorithms, and only had a couple edge cases

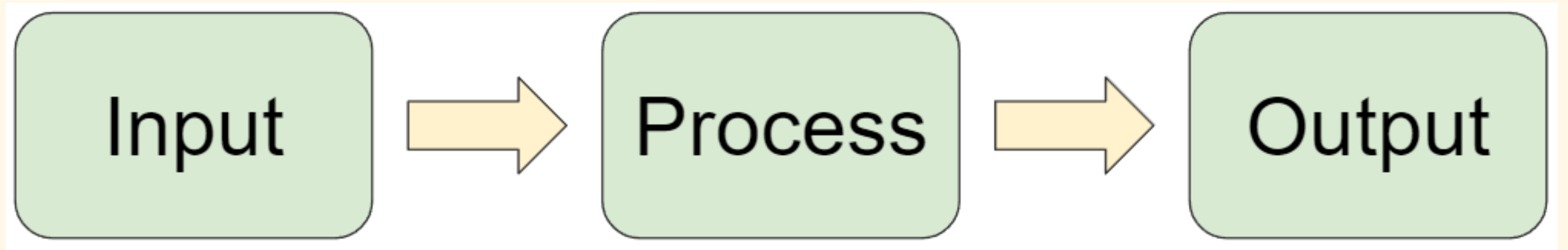
UNIX Philosophy

“Do one thing and do it well”

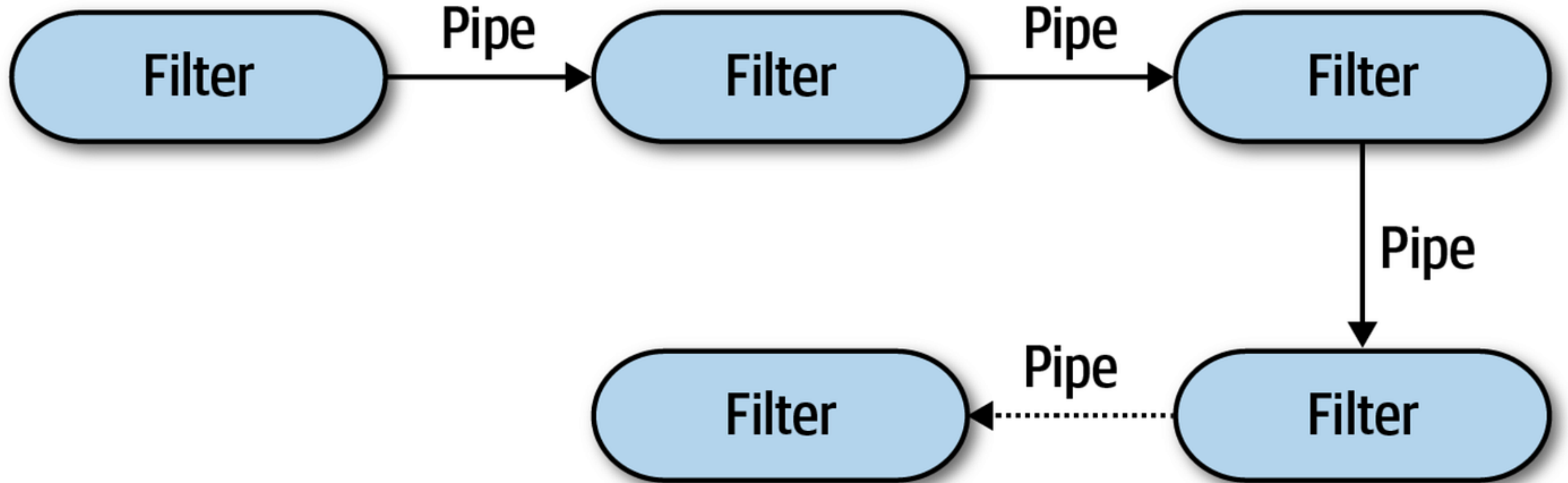
McIlroy responded with this 10-line script:

```
tr -cs A-Za-z '\n' |  
tr A-Z a-z |  
sort |  
uniq -c |  
sort -rn |  
sed ${1}q
```

Data Processing



Data Processing



Activity



Scenario

As an IT analyst, you are given a file *access.log* which contains request data sent to and from your company's web server.



```
user@example:~/CS151$ head -n 3 access.log
```

```
35.187.132.245 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "http://54.236.11.149" "Mozilla/5.0 (Windows; U; MSIE 9.0; Windows NT 9.0; en-US) AppEngine-Google; (+http://code.google.com/appengine; appid: s~virustotalcloud)"
204.13.201.138 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "http://www.bing.com" "Mozilla/5.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; .NET CLR 1.0.3705; .NET CLR 1.1.4322)"
64.233.172.114 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/49.0.2623.75 Safari/537.36 Google Favicon"
```

Scenario

You want to quickly answer some questions based upon this data:

1. How many total requests were made?
2. Of these requests, how many of them were specifically GET requests?
3. How many unique IP addresses can be found in the file?

Scenario

You want to quickly answer some questions based upon this data:

4. Which IP address made the most requests (occurred the most in the log file)?
5. How many requests were made by the IP address 65.154.226.109?
6. Can you list out all the GET endpoints that were attempted to be accessed?

Scenario

Challenge

How many unique status codes were returned by the server?

How many attempts were made to exploit CVE-2020-8515?

Filter out logs for a given time-range (11:55:17 – 17:55:23)

Scenario

A partially implemented *script.sh* file is created to help in solving these questions, the first couple questions are done for you:

```
#!/bin/bash

# Get total number of requests
function total_requests() {
    wc -l access.log | sed 's/access.log//g'
}

# Get total number of get requests
function get_requests() {
    grep "GET" access.log | wc -l
}

# Get unique IP Addresses
function get_uniq_ip_addresses() {
    cut -d ' ' -f 1 access.log | sort | uniq | wc -l
}
```

Scenario

You are tasked with implementing these functions:

```

# Find the IP address with the highest occurrence
function get_highest_occurrence_ip() {
    # Delete the colon (:), and insert your command here
    :
}

# Find the number of requests made from ip 65.154.226.109.
function number_of_requests_from_ip() {
    # Delete the colon (:), Insert your command here
    :
}

# List out all GET endpoints that were attempted to be accessed
# Example: GET / => The endpoint is /
#          GET /hudson => The endpoint is /hudson
function list_endpoints() {
    # Delete the colon (:), Insert your command here
    :
}
```

Scenario

Note: To run the script, you must modify the permissions using chmod.



```
user@example:~/CS151$ ./script.sh
-bash: ./script.sh Permission Denied

user@example:~/CS151$ chmod +x script.sh

user@example:~/CS151$ ./script.sh
Total Requests
156

Number of GET Requests
127
...
```

Helpful Commands



man

Short for “manual”

Provides a quick reference on how to use a command.

```
MAN(1)                                Manual pager utils                                MAN(1)

NAME
    man - an interface to the system reference manuals

SYNOPSIS
    man [man options] [section] page ...] ...
    man -k [apropos options] regex ...
    man -K [man options] [section] term ...
    man -f [whatis options] page ...
    man -l [man options] file ...
    man -w|-W [man options] page ...

DESCRIPTION
    man is the system's manual pager. Each page argument given to man is normally the name of a program, utility or function. The manual page associated with each of these arguments is then found and displayed. A section, if provided, will direct man to look only in that section of the manual. The default action is to search in all of the available sections following a pre-defined order (see DEFAULTS), and to show only the first page found, even if page exists in several sections.

    The table below shows the section numbers of the manual followed by the types of pages they contain.

    1 Executable programs or shell commands
    Manual page man(1) line 1 (press h for help or q to quit)
```

head

Outputs the first part of the file.

Useful Flags

- n [number of lines] – Outputs the first n lines of the file
- c [number of bytes] – Outputs the first c bytes of the file



```
user@example:~/CS151$ head -n 3 access.log
```

```
35.187.132.245 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "http://54.236.11.149" "Mozilla/5.0 (Windows; U; MSIE 9.0; Windows NT 9.0; en-US) AppEngine-Google; (+http://code.google.com/appengine; appid: s~virustotalcloud)"
204.13.201.138 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "http://www.bing.com" "Mozilla/5.0 (compatible; MSIE 8.0; Windows NT 6.1; Trident/4.0; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; .NET CLR 1.0.3705; .NET CLR 1.1.4322)"
64.233.172.114 - - [02/May/2020:00:05:32 +0000] "GET / HTTP/1.1" 200 18 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/49.0.2623.75 Safari/537.36 Google Favicon"
```

tail

Outputs the last part of the file.

Useful Flags

- n [number of lines] – Outputs the last n lines of the file
- c [number of bytes] – Outputs the last c bytes of the file

```
user@example:~/CS151$ tail -n 3 access.log
```

```
45.10.234.77 - - [02/May/2020:22:11:39 +0000] "GET / HTTP/1.1" 200 18 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 2.0.50727)"
109.194.19.200 - - [02/May/2020:23:00:25 +0000] "GET / HTTP/1.1" 200 18 "-" "Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/51.0.2704.103 Safari/537.36"
85.17.27.40 - - [02/May/2020:23:15:37 +0000] "GET / HTTP/1.0" 200 18 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/72.0.3626.121 Safari/537.36"
```

grep

Print out lines that match a provided pattern.

Useful Flags

- i – Ignore Case
- v – Invert matches, i.e. exclude these results
- c – Print a count instead of the lines

cut

Remove sections from each line of a given input/file.

Useful Flags

- d – Delimiter, separate lines based on the provided character
- f [fields] – Select only the provided fields

sort

Sorts a given input alphabetically/numerically.

Useful Flags

- n – Sort based on string numeric value
- r – Reverse order
- R – Shuffle the results

uniq

Removes duplicate lines.

!!! Important – Input must be sorted **first**.

Useful Flags

- c – Provide a count for each duplicated entry

sed

“Stream editor”, used to transform/filter text.

Format

Find and replace:

```
sed 's/old/new/g' [input-file]
```

References: <https://quickref.me/sed.html>

tr

Translate/Delete Characters

Examples

`tr [:lower:] [:upper:]` – Converts all lowercase letters to upper case

`tr -d '[character]'` – Removes all characters inside the quotes

`tr -s` – Squeeze, remove duplicate characters